

Proyecto Final – Motor de Ajedrez

Avances - 2

José Antonio Mérida Castejón

20 de mayo de 2026

1. Descripción General

Este informe documenta las variantes implementadas del motor de ajedrez, partiendo de una base común (generación de movimientos, representación del tablero mediante bitboards y protocolo UCI) sobre la cual se implementaron y compararon distintas técnicas de búsqueda y evaluación. Cada variante se mantiene en una rama (*branch*) independiente del repositorio, permitiendo comparaciones directas mediante partidas automatizadas.

La infraestructura base fue adaptada del motor de código abierto *Reckless* (Rust), del cual se reutilizó la representación de tablero, generación de movimientos legal, manejo del protocolo UCI y la tabla de transposiciones. La evaluación neuronal original fue eliminada y reemplazada por las técnicas descritas a continuación.

2. Punto de Partida: Infraestructura Común

Todas las variantes comparten los siguientes componentes:

- **Representación del tablero:** Bitboards de 64 bits por pieza y color. Permite operaciones de conjuntos sobre casillas en tiempo constante.
- **Generación de movimientos:** Generación legal separada en movimientos ruidosos (*capturas y promociones*) y silenciosos, con soporte para en passant y enroques.
- **Búsqueda en profundización iterativa:** Se busca a profundidad 1, luego 2, y así sucesivamente hasta agotar el tiempo. El resultado de cada iteración alimenta la siguiente mediante la tabla de transposiciones.
- **Búsqueda alpha-beta (negamax):** Poda de ramas que no pueden afectar el resultado. Base de toda la búsqueda.
- **Búsqueda de quietud (*quiescence search*):** Al llegar a profundidad cero, en lugar de evaluar estáticamente, se continúa buscando solo capturas hasta que la posición sea estable. Evita el efecto horizonte: que el motor no vea una recaptura inmediata tras el último movimiento analizado.

- **Tabla de transposiciones:** Caché indexada por hash Zobrist que almacena resultados de posiciones ya buscadas. Permite evitar búsquedas redundantes en posiciones alcanzadas por distintos órdenes de movimientos.
- **PVS (*Principal Variation Search*):** Optimización sobre alpha-beta: el primer movimiento se busca con ventana completa; los demás se buscan primero con ventana nula (un punto). Si alguno supera alpha, se hace una nueva búsqueda completa para confirmarlo. Reduce el trabajo total sin afectar la exactitud.
- **Detección de tablas:** Por regla de 50 movimientos, repetición de posición y material insuficiente (R vs R, R+A vs R, R+C vs R, alfiles de mismo color).

3. Variantes Implementadas

1. v1-base — Alpha-Beta básico sin ordenamiento

La variante más simple. Implementa alpha-beta con profundización iterativa y una función de evaluación heurística basada en:

- **Material:** valores fijos por tipo de pieza (Peón 100, Caballo 320, Alfil 330, Torre 500, Dama 900).
- **Tablas pieza-casilla (PST):** bonificaciones posicionales por casilla según la *Simplified Evaluation Function* de Tomasz Michniewski. Una única tabla por pieza, independiente de la fase del juego.

Los movimientos se generan y prueban en el orden que produce el generador, sin ningún tipo de ordenamiento. Sirve como línea base mínima.

2. v1.5-move-ordering — Ordenamiento de movimientos

Sobre la misma evaluación, se agrega un sistema de ordenamiento de movimientos por etapas (*staged move ordering*):

- **Movimiento de la tabla de transposiciones:** el mejor movimiento de la búsqueda anterior se intenta primero. Al ser generalmente el mejor movimiento, produce cortes beta inmediatos y poda el resto del árbol.
- **Capturas buenas (SEE):** las capturas se filtran mediante *Static Exchange Evaluation*, que determina si una secuencia de capturas en una casilla resulta en ganancia material. Las capturas perdedoras se posponen.
- **Movimientos silenciosos con historial:** movimientos no capturantes puntuados por tablas de historial que aprenden qué movimientos generaron cortes beta en búsquedas anteriores. Incluye historial de quietud, historial ruidoso e historial de continuación (pares de movimientos consecutivos).
- **Capturas malas:** las capturas que pierden material según SEE se intentan al final.

El ordenamiento no afecta la exactitud del resultado, solo la eficiencia: alpha-beta poda más ramas cuando ve movimientos buenos primero.

3. v2-tapered-eval — Evaluación graduada (PeSTO)

Se reemplaza la función de evaluación con las tablas de *PeSTO* (*Piece-Square Tables Optimization*), que introducen evaluación graduada (*tapered evaluation*):

- **Dos tablas por pieza:** una para apertura/mediojuego y otra para finales, con valores de material distintos en cada fase (Peón MG: 82 / EG: 94, Dama MG: 1025 / EG: 936, etc.).
- **Fase de juego:** calculada sumando el material remanente (Dama=4, Torre=2, Alfil=1, Caballo=1; máximo=24). A medida que caen piezas, la evaluación transiciona suavemente hacia los valores de final.
- **Interpolación:** $score = (mg_score \times fase + eg_score \times (24 - fase)) / 24$.

El impacto más visible es el rey: en mediojuego recibe penalización por centralizarse, en final recibe bonificación. Los peones avanzados también valen mucho más en tablas de final.

4. v2.1-nmp — Poda de Movimiento Nulo

Se agrega *Null Move Pruning* (NMP) a la búsqueda:

La idea central es que si nuestra posición es tan buena que incluso si le dáramos al rival un movimiento gratis sigue sin poder superar beta, entonces podemos podar sin buscar más. Se implementa pasando el turno (*null move*), buscando a profundidad reducida ($R = 3 + \text{profundidad}/3$) con ventana nula, y si el resultado sigue siendo $\geq \beta$, se retorna beta.

Se desactiva cuando:

- El lado a mover está en jaque (no se puede pasar el turno en jaque).
- El movimiento anterior también fue nulo (evitar cadenas de movimientos nulos).
- El lado a mover solo tiene peones y rey (riesgo de *zugzwang*: posiciones donde pasar el turno empeora la posición).
- La evaluación estática no supera beta (la posición no es suficientemente buena).

5. v2.2-lmr — Reducción de Movimientos Tardíos

Se agrega *Late Move Reductions* (LMR):

Los movimientos ordenados al final de la lista (tras las capturas, el movimiento TT, y los mejores silenciosos) probablemente sean malos. En vez de buscarlos a profundidad completa, se reducen: se buscan a profundidad $d - 1 - R$ donde $R = \lfloor \ln(\text{profundidad}) \times \ln(n_movimiento) / 2 \rfloor$. Si el resultado supera alpha, se rehace la búsqueda a profundidad completa para confirmar.

Solo se aplica a movimientos silenciosos, después del tercer movimiento, y cuando la profundidad es ≥ 3 . No reduce capturas ya que estas son más forzadas y difíciles de descartar sin buscarlas.

6. v2.3-rfp — Poda de Futilidad Inversa

Se agrega *Reverse Futility Pruning* (RFP), también llamada poda estática de movimiento nulo:

Si la evaluación estática supera beta por un margen de $75 \times$ profundidad, es muy improbable que cualquier movimiento haga caer la puntuación por debajo de beta. Se retorna la evaluación estática sin buscar. Solo se aplica en nodos no-PV, fuera de jaque, y a profundidad ≤ 8 .

Es la inversa de la poda de futilidad clásica: en lugar de podar cuando somos demasiado malos para alcanzar alpha, podamos cuando somos demasiado buenos para caer por debajo de beta.

7. v2.4-additional-heuristics — LMP y Extensiones Singulares

Se agregan dos técnicas adicionales:

Poda de Movimientos Tardíos (LMP): a profundidad ≤ 4 , si ya se intentaron suficientes movimientos silenciosos (umbral: 8/12/16/20 según profundidad), los restantes se omiten completamente en vez de reducirse. Más agresivo que LMR para nodos superficiales.

Extensiones Singulares: antes del bucle de movimientos, si el movimiento de la tabla de transposiciones tiene profundidad suficiente y es una cota inferior, se buscan todos los *demás* movimientos a media profundidad con una ventana justo por debajo del puntaje TT. Si ninguno supera ese umbral, el movimiento TT es “singular” (claramente el mejor) y se le otorga 1 ply adicional de búsqueda. Permite al motor profundizar en líneas tácticas críticas que de otro modo quedarían en el horizonte.

4. Pendiente: Evaluación NNUE

La siguiente etapa consiste en reemplazar la función de evaluación heurística por una red neuronal eficiente para actualizaciones incrementales (*Efficiently Updatable Neural Network*, NNUE). Esta arquitectura, introducida por Nasu (2018) y popularizada por Stockfish a partir de 2020, permite mantener velocidades de inferencia comparables a la evaluación heurística mediante actualizaciones parciales al realizar un movimiento.

El entrenamiento se realizará mediante destilación de conocimiento sobre posiciones anotadas con evaluaciones de Stockfish.

5. Resultados Iniciales

Se probó la versión v2.4 con todas las heurísticas implementadas, utilizando Stockfish a diferentes ratings. Se probaron ratings más pequeños (1500–2000) incrementando hasta tener una serie de juegos más parejos.

Prueba 1: Reckless vs. Stockfish 2300 ELO (8+0.08, 1t, 16MB)

Métrica	Valor
Partidas	100
Victorias / Derrotas / Tablas	91 / 9 / 0
Puntos	91.0 (91.00 %)
Elo	401,92 ± 128,33
nElo	524,36 ± 68,10
LOS	100.00 %
DrawRatio	18.00 %
PairsRatio	∞
Ptnml (0–2)	[0, 0, 9, 0, 41]
Tiempo total	00:09:35

Cuadro 1: Resultados contra Stockfish 2300 ELO. El engine se ubica en aproximadamente 2700+ ELO.

Prueba 2: Reckless vs. Stockfish 2700 ELO (8+0.08, 1t, 16MB)

Métrica	Valor
Partidas	100
Victorias / Derrotas / Tablas	45 / 49 / 6
Puntos	48.0 (48.00 %)
Elo	−13,90 ± 65,35
nElo	−14,68 ± 68,10
LOS	33.64 %
DrawRatio	46.00 %
PairsRatio	0.80
Ptnml (0–2)	[11, 4, 23, 2, 10]
Tiempo total	00:11:21

Cuadro 2: Resultados contra Stockfish 2700 ELO.

Prueba 3: Reckless vs. Stockfish 2700 ELO con Opening Book (10+0.1, 1t, 16MB, UHO_4060_v4.epd)

Métrica	Valor
Partidas	400
Victorias / Derrotas / Tablas	192 / 175 / 33
Puntos	208.5 (52.12 %)
Elo	$14,77 \pm 31,23$
nElo	$16,17 \pm 34,05$
LOS	82.40 %
DrawRatio	48.00 %
PairsRatio	1.26
Ptnml (0-2)	[36, 10, 96, 17, 41]
WL/DD Ratio	31.00
Tiempo total	00:13:37

Cuadro 3: Resultados con opening book UHO_4060_v4.epd. ELO estimado: ≈ 2615 .